Prisoner's Dilemma Practice App

A short guide for students. Write a strategy, practise it, and enter the class tournament.

1. Get the app

1. Download the file for your computer from the Releases page of the ipd-app repository. That is `ipd-app-windows.exe`, `ipd-app-macos` or `ipd-app-linux`.
2. Double click it. Your browser opens at `http://127.0.0.1:5000`. Keep the black window open while you work. Close it to stop.
3. Open the Settings tab, type the website address and class token your instructor gave you, and press Connect. You only do this once.

> Windows may say "Windows protected your PC". Click More info, then Run anyway. On a Mac, see the README for the one line you need to run the first time.

2. Your first strategy

A strategy is a Python function. The app calls it every round and it returns `'C'` to cooperate or `'D'` to defect. This one is Tit-for-Tat. It cooperates first and then copies the opponent.

```
def tit_for_tat(last_moves, my_history, opponents_histories, meta):
    if not last_moves:
        return 'C'
    return last_moves[0]
```

Input	What it holds
`last_moves`	Each opponent's move from the last round, such as `['C']`. Empty in the first round.
`my_history`	Every move you have made so far.
`opponents_histories`	One list of moves for each opponent.
`meta`	Extra information about the game. See the next section.

Rules to remember

- Always return exactly `'C'` or `'D'`. Anything else, a crash or an endless loop loses you the match.
- The name you type in the app is only a label. It must be unique, and it does not need to match your function name.
- Files, the network, `exec`, `eval` and `open` are blocked.

3. Using meta

`meta` is a dictionary with five things in it. Each example below is a complete strategy that works in all three games.

meta['round']

The round number, starting at 0.

```
def slow_start(last_moves, my_history, opponents_histories, meta):
    # Be friendly for the first 5 rounds, then copy the opponent.
    if meta['round'] < 5:
        return 'C'
    return last_moves[0]
```

meta['rng']

A random number generator. Use it instead of `import random`, so a run with a seed can be repeated exactly.

```
def generous_tit_for_tat(last_moves, my_history, opponents_histories, meta):
    # Like Tit-for-Tat, but forgive a defection about 1 time in 10.
    if not last_moves or last_moves[0] == 'C':
        return 'C'
    if meta['rng'].random() < 0.1:
        return 'C'
    return 'D'
```

meta['n_players']

How many players are in the game, counting you. It is 2 in 1v1, the group size in N-Player and the number of cores in the OS Simulation.

```
def careful_in_crowds(last_moves, my_history, opponents_histories, meta):
    # In a big group, only cooperate if nobody defected last round.
    if not last_moves:
        return 'C'
    if meta['n_players'] > 2:
        return 'C' if 'D' not in last_moves else 'D'
    return last_moves[0]
```

meta['player_index']

Your seat in the game, starting at 0. You will rarely need it.

```
def seat_based_start(last_moves, my_history, opponents_histories, meta):
    if not last_moves:
        return 'C' if meta['player_index'] % 2 == 0 else 'D'
    return last_moves[0]
```

meta['tournament_info']

Information about the game you are in. Its `format` is `'2-player'` in 1v1, `'n_player'` in N-Player and `'os_simulation'` in the OS Simulation.

```
def adapts_to_the_game(last_moves, my_history, opponents_histories, meta):
    game = meta['tournament_info'].get('format')
    if not last_moves:
        return 'C'
    if game == 'os_simulation':
        # last_moves[0] is 'D' when the computer is busy. Give way when it is busy.
        return 'C' if last_moves[0] == 'D' else 'D'
    if game == 'n_player':
        return 'C' if last_moves.count('C') * 2 >= len(last_moves) else 'D'
    return last_moves[0]
```

4. The three games

1v1

Every strategy plays every other strategy, one pair at a time. You are ranked by a weighted score made of win rate, cooperation and points. Move the sliders to decide how much each part counts.

N-Player

Everyone plays in one group and moves at the same time. `last_moves` then has one move for each other player. Choose how the group is paid, for example Public Goods, where cooperators pay into a pot that is multiplied and shared.

OS Simulation

Your strategy becomes a CPU scheduler. Cooperating means giving up the CPU and defecting means keeping it. `last_moves[0]` tells you whether the computer is busy (`'D'`) or calm (`'C'`). Each strategy is run 10 times on different task sets and averaged. The plus or minus range next to each number shows how sure the result is. If two ranges overlap, neither strategy is clearly better.

5. Practise against the class

- Press Refresh Gallery to fetch your classmates' strategies. Tick the ones you want to face.
- Press Save locally to keep a strategy on your computer.
- On the Hub tab, import a classmate's strategy and press Open in Practice to load it into an editor.
- Tick Run in sandbox mode to test strategies you did not write in a safe, separate process first.

6. Submit

1. Open the Hub tab and paste your strategy under Submit your strategy to the website.
2. Press the button. The app checks and test plays your strategy first and tells you about any problem.
3. The class website opens with your strategy filled in. Log in and press Submit. You can update it any time before the tournament.

7. Common mistakes

- A branch of your code that never returns. The function then returns `None` and fails.
- Reading `last_moves[0]` in the first round, when the list is still empty. Check `if not last_moves:` first.
- Using `random.random()`. Use `meta['rng']` instead so results can be repeated.
- Two entries with the same name in one run. Rename one of them.

Good luck, and have fun.